

3D GS 实现步骤:

1. 原始照片拍摄 (原始数据)
2. SfM 构建了点云 (同 Nerf)
3. 3DGS 算法优化, 拟合高斯 (调整)
4. 场景构建 (Scene representation)
5. 新视角域 (投影)

(注意 3DGS 无需 MVS, 这也是个优势)

为什么要注意高斯投影的先后顺序?
 * 因为运用 α -blending, 顺序不一样, 大副影响结果。

3DGS 是怎么进行高斯优化的?

增删查改。

缺少的高斯要增, 多的要删, 查线有问题的高斯进行修改。

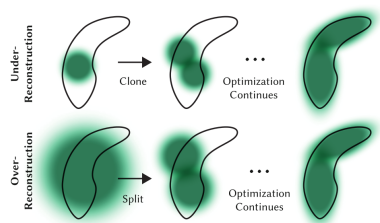


Fig. 4. Our adaptive Gaussian densification scheme. Top row (under-reconstruction): When small-scale geometry (black outline) is insufficiently covered, we clone the respective Gaussian. Bottom row (over-reconstruction): If small-scale geometry is represented by one large splat, we split it in two.

3DGS 作者创造的 GPU 为树器厉害在那?

1. tile-based. 把 2D 平面分成了不同的 tile
2. 渲染时不设置固定前 K 个高斯, 同时设置了一个阈值, 防止每个 tile 存储了过大的高斯列表。

3DGS 三项核心贡献:

3D 高斯表示场景, 自适应优化高斯, 实时渲染高斯。
 (表示创新, 训练创新, 渲染创新)

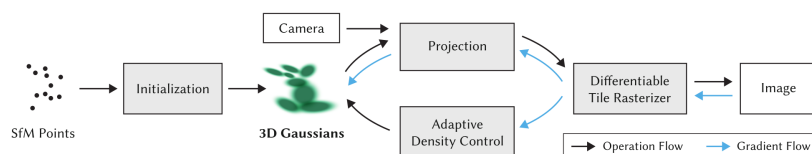


Fig. 2. Optimization starts with the sparse SfM point cloud and creates a set of 3D Gaussians. We then optimize and adaptively control the density of this set of Gaussians. During optimization we use our fast tile-based renderer, allowing competitive training times compared to SOTA fast radiance field methods. Once trained, our renderer allows real-time navigation for a wide variety of scenes.

注意 3DGS 的表树仍有些不一样。
 传统为树化直接计算像素中心在不在图像里, 在就把整个像素涂上颜色, 而 3DGS 中运用的表树化为什么不一样。他们采用渲染, 只要渲染的格子就算数。如果格子中心不在图像里, 就按距离整个权重, 再将权重决定颜色的不透明度。这样就使得生成的图形边缘不会锯齿化。